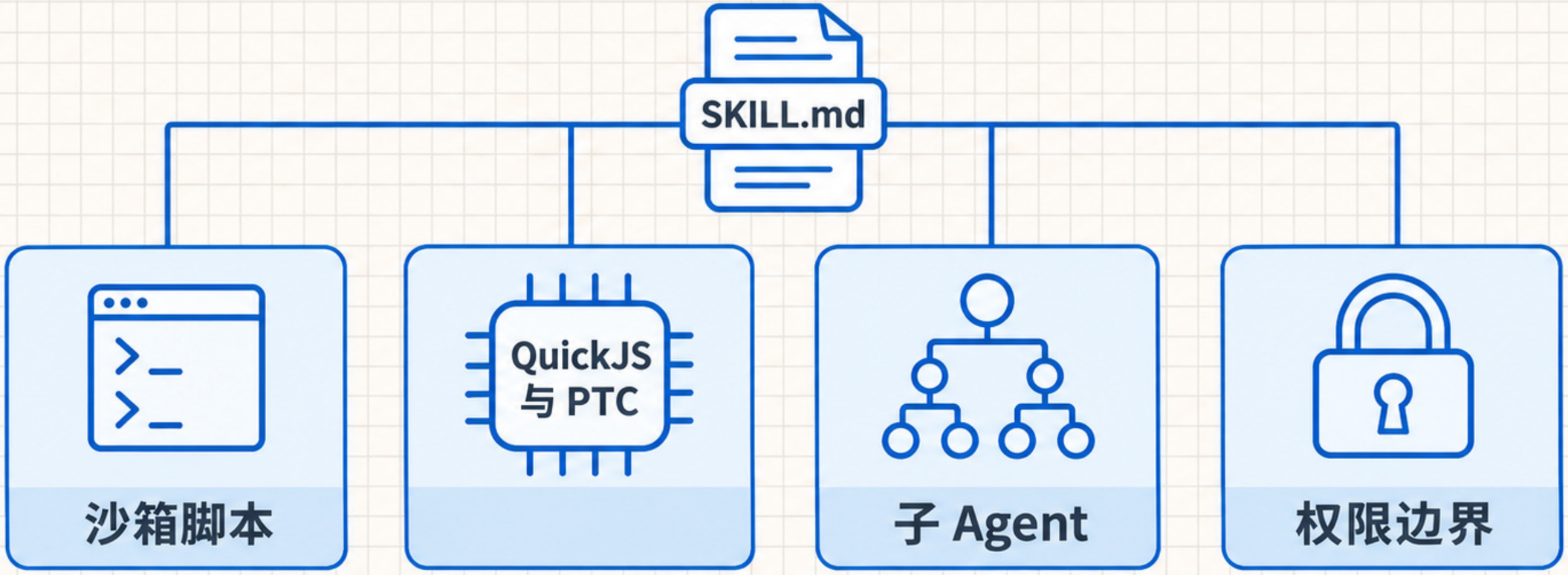
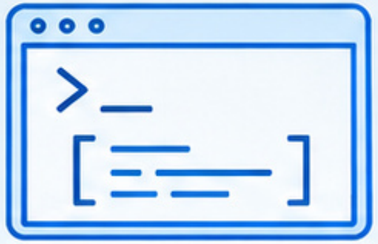
Deep Agents 实战

⚠ 第 7 章 · 下篇: Skills 高级用法



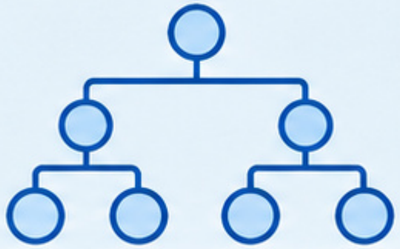
本篇地图：从会用到用对

代码执行



⚠ 读取说明与执行代码分开

子 Agent



⚠ 继承规则不等于文件隔离

SKILL.md

解释器



⚠ eval 与 PTC，不自动导入 Skill

权限



⚠ 操作规则、人工确认与共享分层

读取 Skill，不等于执行 Skill

沙箱脚本



配置执行后端

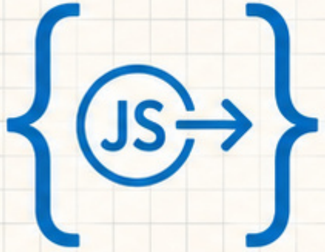
↓

同步脚本与依赖

↓

运行 scripts/search.py

QuickJS



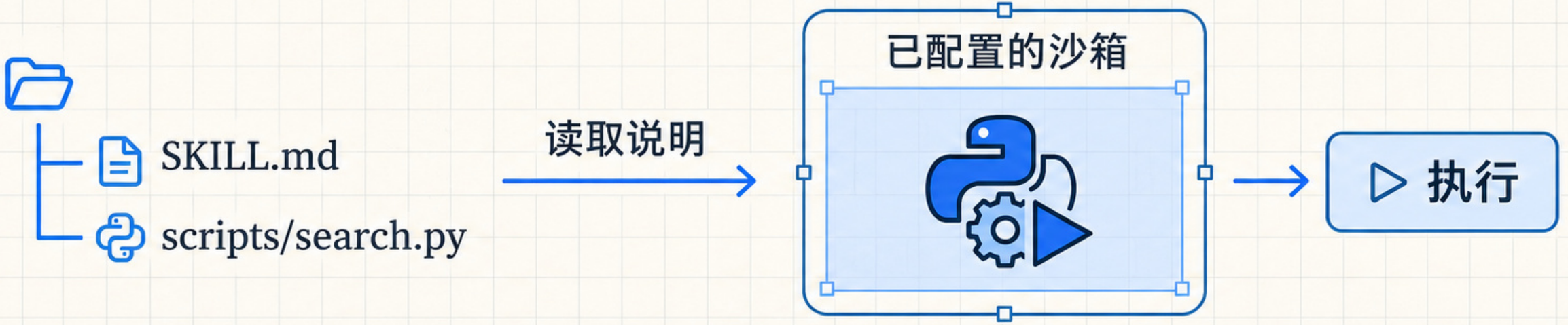
eval 运行 JS/TS

↓

PTC 调用白名单工具

⚠ | skills 配置只负责发现与说明，不会自动导入或执行脚本。

沙箱脚本：运行环境也要准备好



- 脚本必须位于执行环境中
- 依赖、网络与工作目录需显式配置
- 与主进程隔离不代表没有外部副作用

⚠ 可使用 Daytona 等执行后端；读文件后端本身不自动提供 Shell。

脚本调用：明确路径与输入

技能目录：/skills/arxiv-search/
说明文件：SKILL.md
实现文件：scripts/search.py

明确执行环境中的路径

示例命令（脚本已部署时）：

```
python /skills/arxiv-search/scripts/search.py "agent" --max-results 10
```

查询词作为参数

脚本负责调用 API 与整理结果。

脚本负责调用 API 与整理结果

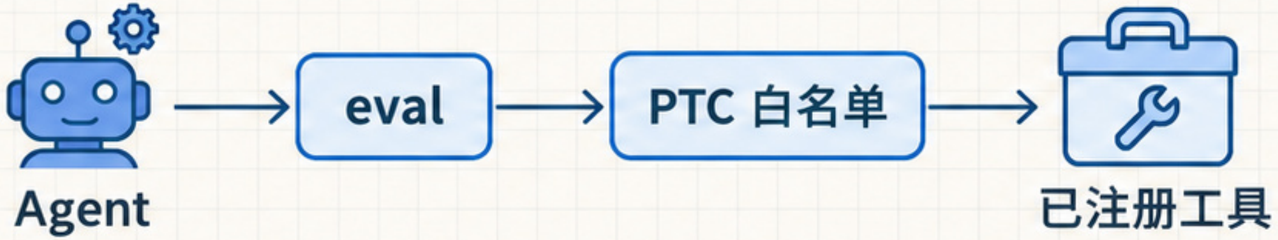
⚠ 路径是示例；需按沙箱的实际挂载位置调整。

QuickJS：用 eval 与 PTC 复用逻辑

步骤一：用 `eval` 执行脚本



步骤二：通过 `PTC` 复用逻辑



要点

- mode="call"：每次 eval 是全新环境
- skills 不会自动注册解释器函数
- 复用业务逻辑：显式工具或沙箱脚本

⚠ langchain-quickjs 0.3.9 不支持 skills_backend 或旧版 @/skills 导入。

当前解释器接入：最小示例

1. model 需预先配置，需安装 QuickJS 依赖。
2. 每次 eval 不保留上次变量与函数。
3. 如需 PTC，另行配置已注册工具白名单。

```
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware

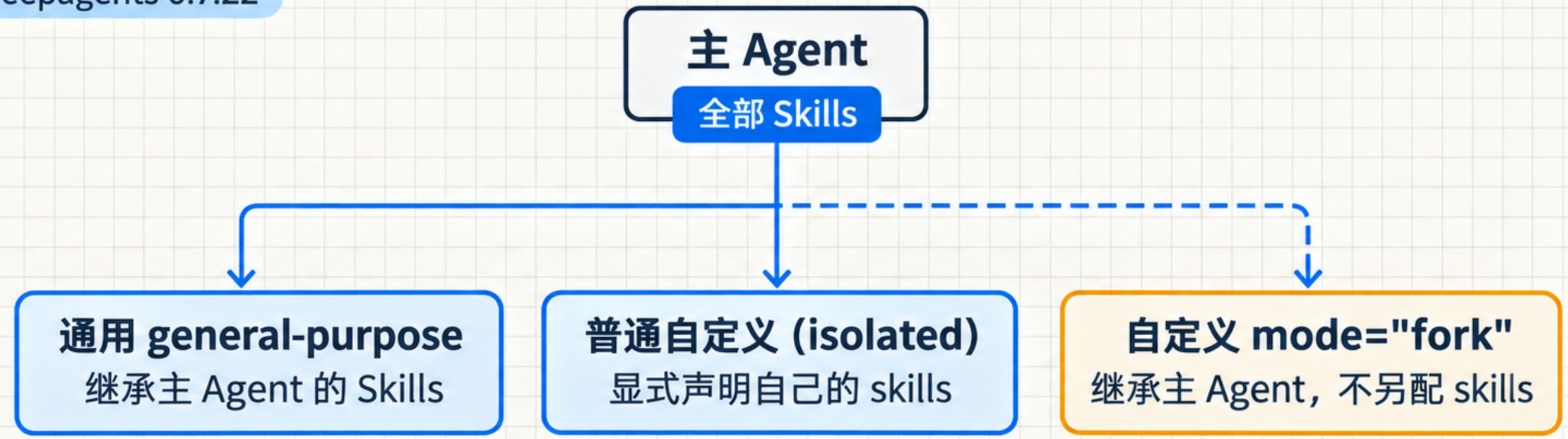
agent = create_deep_agent(
  model=model,
  middleware=[CodeInterpreterMiddleware(mode="call")],
)
```

⚠ 不使用旧 skills_backend、entrypoint 自动导入机制。

Skills 与子 Agent：三种情况

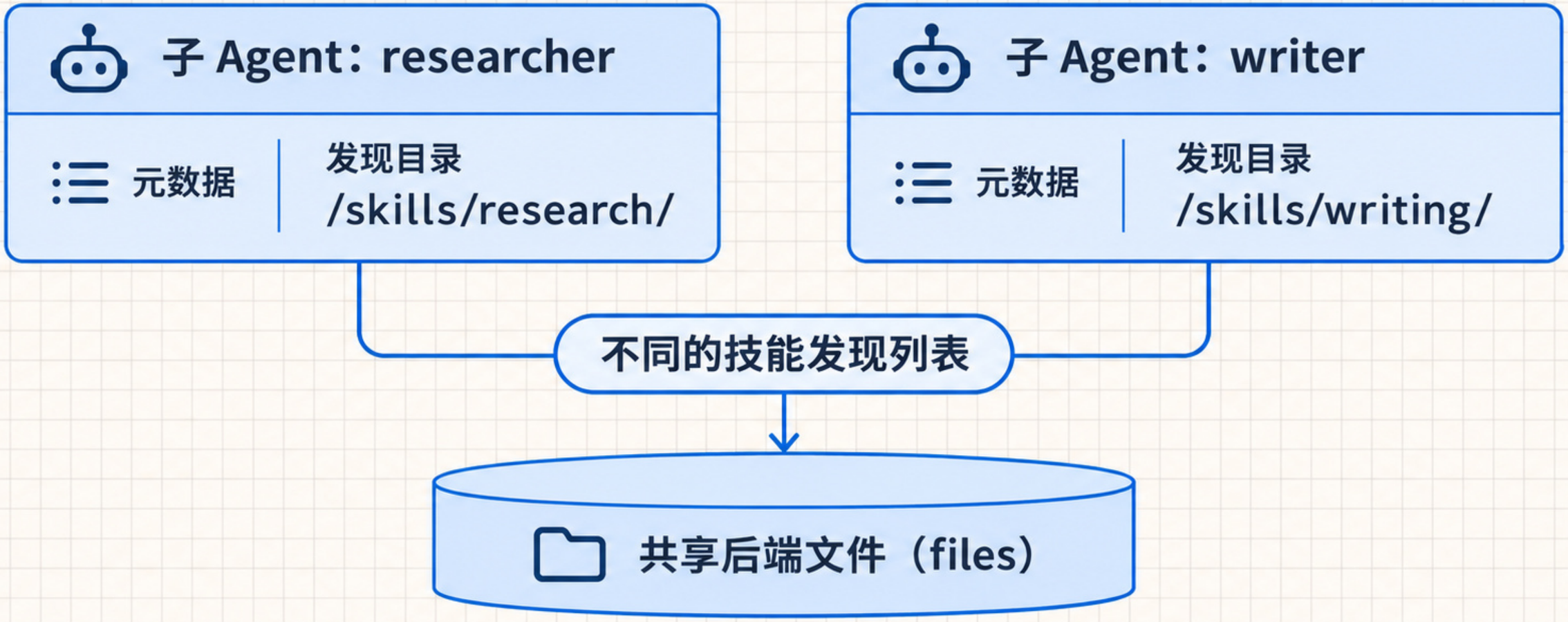
三种分支架构的主 Agent 技能（Skills）继承方式。

deepagents 0.7.22



⚠ 元数据与上下文隔离，不代表后端文件隔离；共享后端仍可能共享文件。

自定义 skills 是发现范围，不是访问权限



⚠ 未列入 skills 的路径，不会因此自动禁止读取。

⚠ 真正的访问边界由后端与权限策略实现。

文件权限：首条匹配，默认允许

1. deny：拒绝匹配的文件工具操作
2. interrupt：暂停等待人工决定
3. allow：允许操作

- 按声明顺序首条匹配；无匹配默认 allow
- interrupt 需 Checkpointer 与线程配置
- 拒绝待执行调用，不是回滚先前副作用。

⚠ 文件工具规则不等于 Shell 或管理员的全局沙箱。

只读技能：把规则真正接入 Agent

model 预先配置；文件已存在。

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import FilesystemBackend

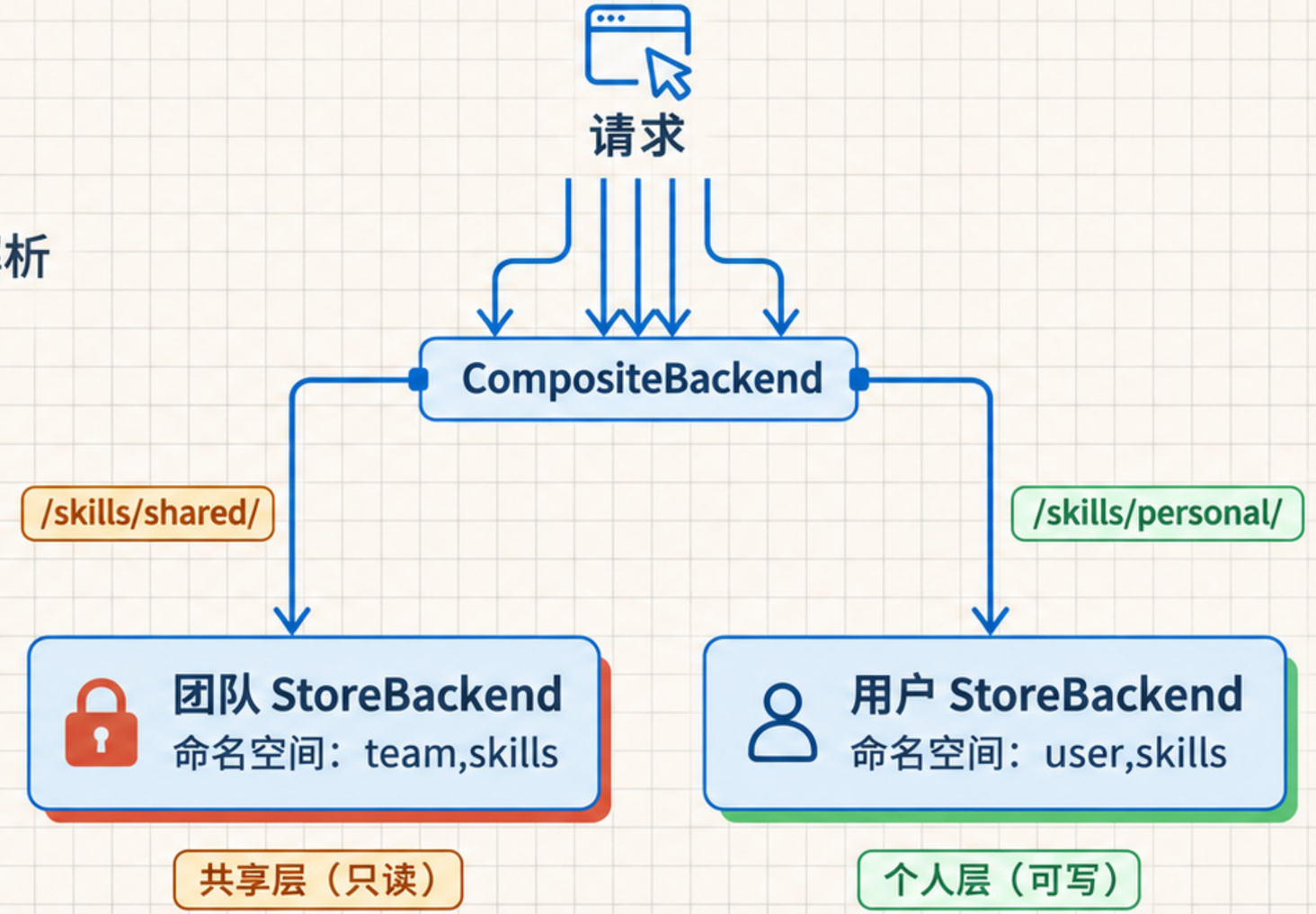
agent = create_deep_agent(
    model=model,
    backend=FilesystemBackend(root_dir="./project", virtual_mode=True),
    skills=["/skills/"],
    permissions=[FilesystemPermission(
        operations=["write"], paths=["/skills/**"], mode="deny"
    )],
)
```



示例保护匹配文件工具写操作；不要把它当作命令执行权限。

共享层 + 个人层：路由与权限分开

- 路由前缀会从存储键剥离
- namespace 必须从可信身份解析
- 共享层只读需独立写权限策略
- 持久化取决于实际 Store 实现



⚠ 0.7.22 对执行型后端的文件权限有组合限制；不能假设 Shell 同受保护。

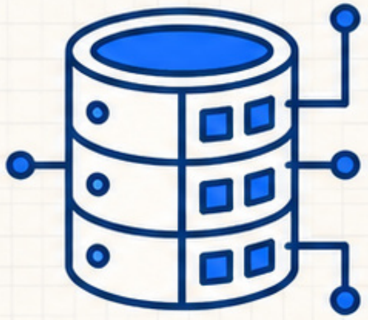
Skills、Memory 与 Tools

Skills



任务方法与可选资源

Memory



可复用的记忆文件与上下文

Tools



执行具体操作的接口

Skills 负责怎么做；Memory 提供已知信息；Tools 负责实际动作。

⚠ 跨会话保留需要相应存储配置。

下一章：长期记忆（Memory）